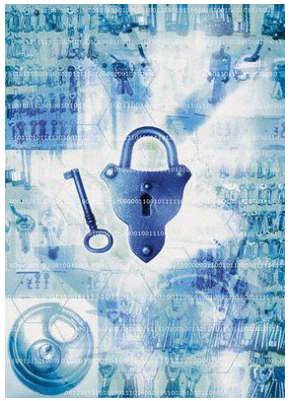
Information Security



Digital signatures

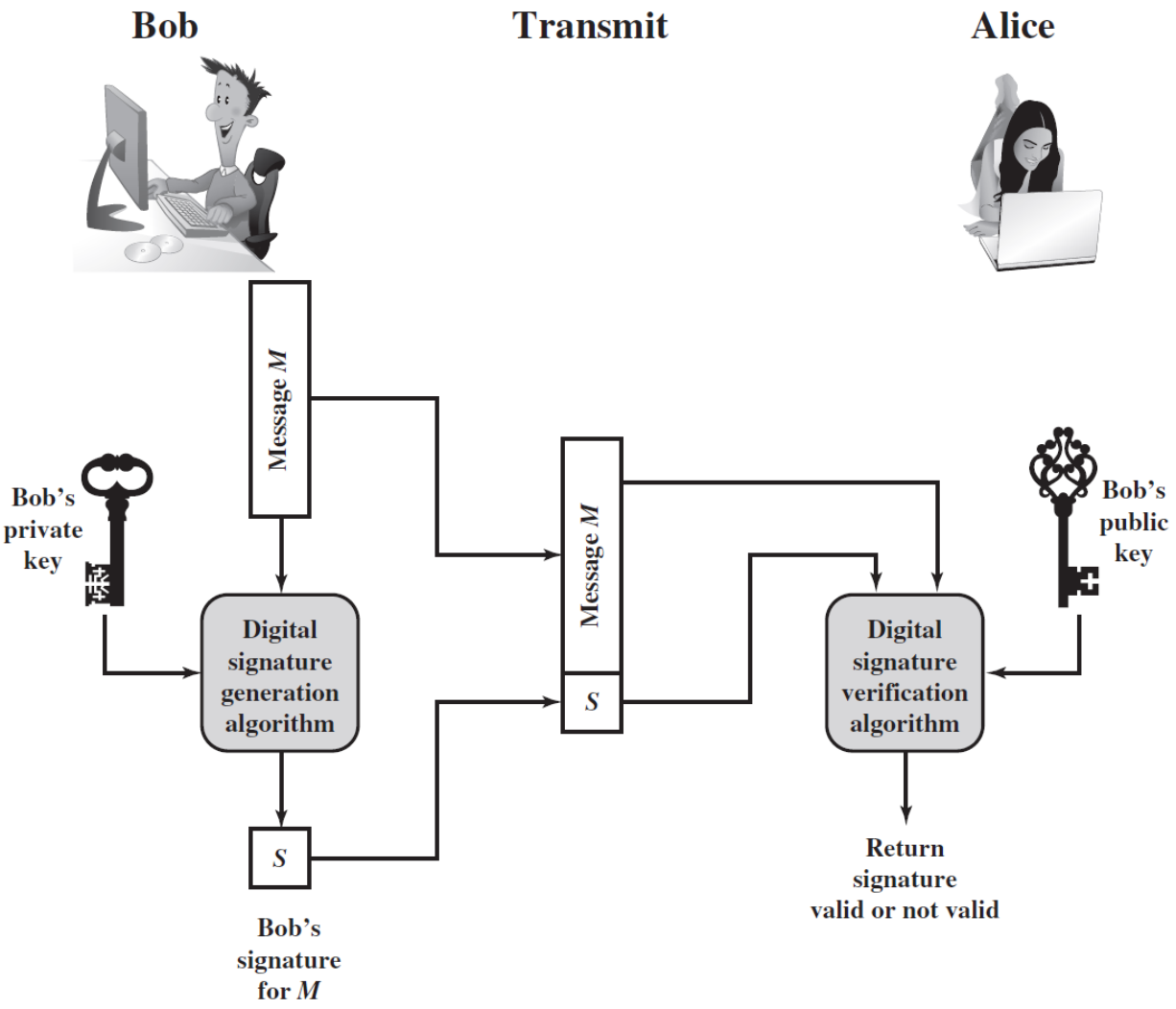
AI컴퓨터공학부
김 희 열

Digital Signature

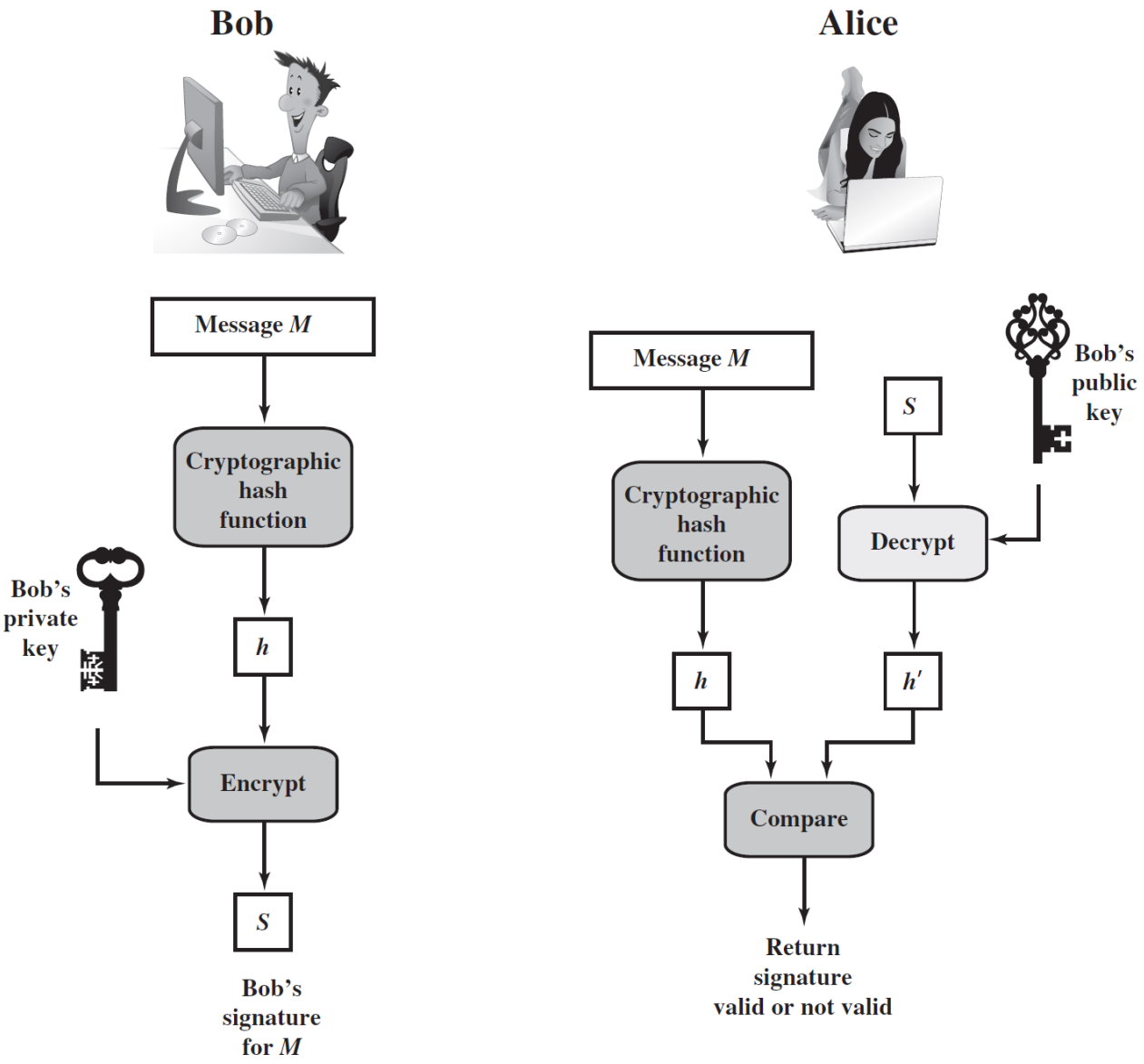
- Message authentication (MAC/HMAC)
  - Protects two parties exchanging messages from any third party
  - Not protects two parties against each other

- Digital signature
  - Solve above problem
  - Required property
    - Must verify the author and the date and time of the signature
    - Must authenticate the contents at the time of the signature
    - Must be verifiable by third parties, to resolve dusputes
  - Digital signature includes authentication

Generic Model of Digital Signature

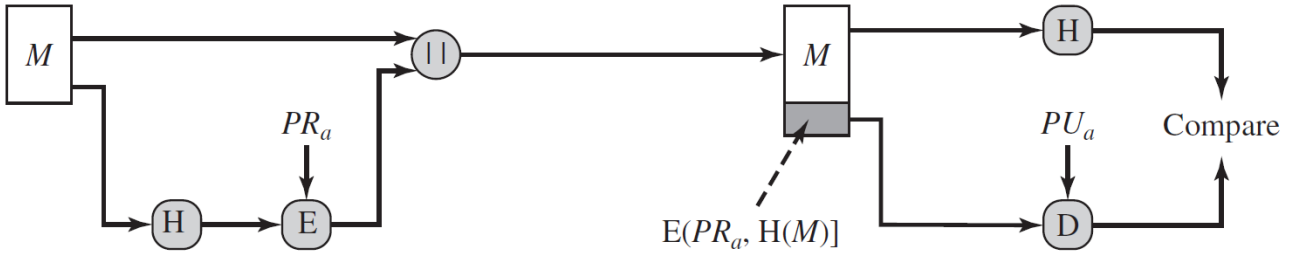


Digital Signature with Hash Function

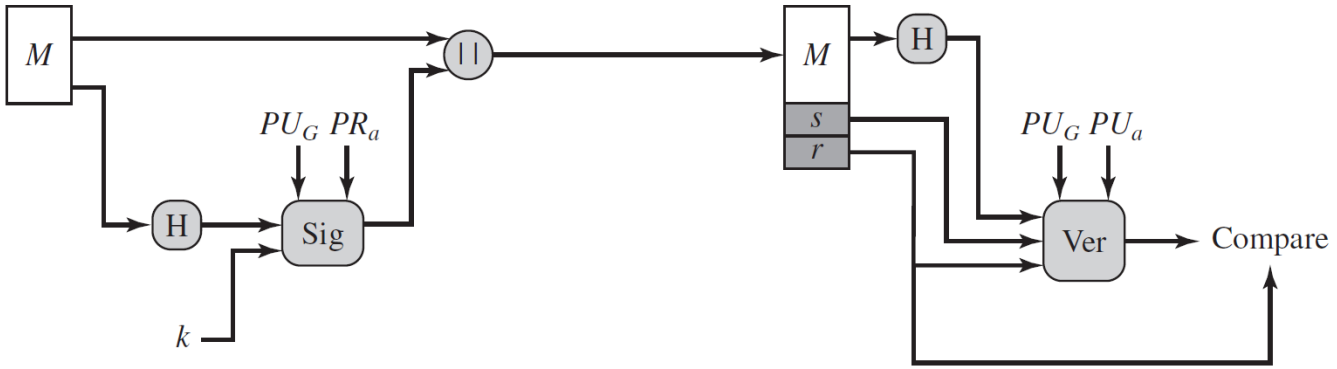


DSA

- Digital Signature Algorithm (DSA) published as FIPS 186 by NIST
- Based on difficulty of discrete logarithm problem
- Based on Elgamal scheme and Schnorr scheme
- Provide only digital signature, not encryption or key exchange



(a) RSA approach



PU_G : global public key

(b) DSA approach

DSA Key

Global Public-Key Components

p prime number where $2^{L-1} < p < 2^L$
for $512 \leq L \leq 1024$ and L a multiple of 64;
i.e., bit length of between 512 and 1024 bits
in increments of 64 bits

q prime divisor of $(p - 1)$, where $2^{N-1} < q < 2^N$
i.e., bit length of N bits

g $= h^{(p-1)/q} \bmod p$
where h is any integer with $1 < h < (p - 1)$
such that $h^{(p-1)/q} \bmod p > 1$

User's Private Key

x random or pseudorandom integer with $0 < x < q$

User's Public Key

$y = g^x \bmod p$

User's Per-Message Secret Number

k random or pseudorandom integer with $0 < k < q$

- Generated per message
- Private key related to PU_G

DSA Signing and Verifying

Signing

$r = (g^k \bmod p) \bmod q$

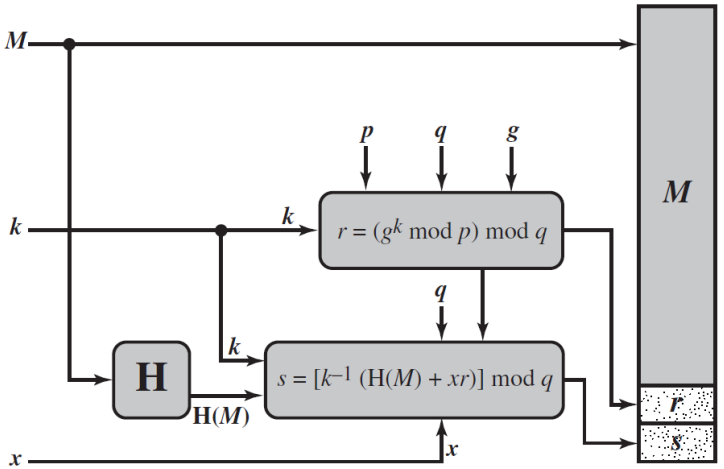
$s = [k^{-1}(H(M) + xr)] \bmod q$

Signature $= (r, s)$

M = message to be signed

$H(M)$ = hash of M using SHA-1

M', r', s' = received versions of M, r, s



Verifying

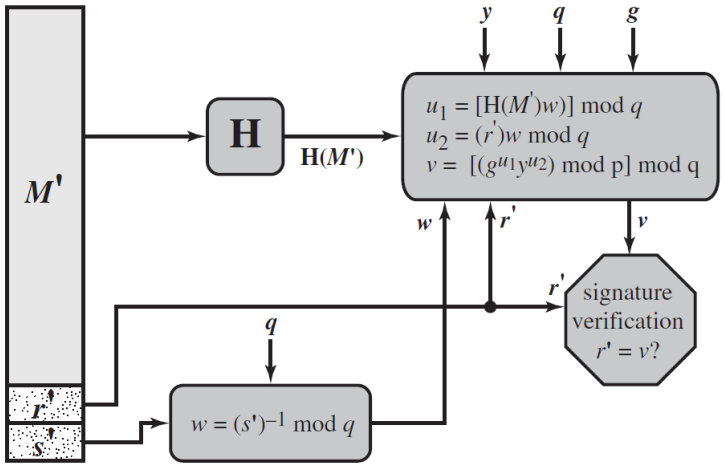
$w = (s')^{-1} \bmod q$

$u_1 = [H(M')w] \bmod q$

$u_2 = (r')w \bmod q$

$v = [(g^{u1} y^{u2}) \bmod p] \bmod q$

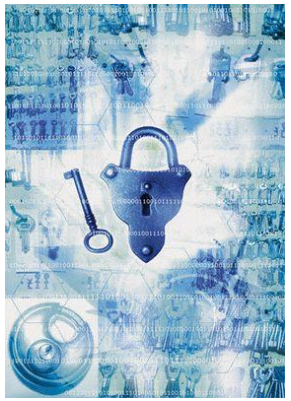
TEST: $v = r'$



DSA

- Infeasible for Trudy to recover k from r, or to recover x from y
  - Discrete Logarithm Problem
- Computing $r = g^k \bmod p$ does not depend on the message
  - Can precompute many values before signing many documents
- Bit length (L, N) pair
  - (1024, 160), (2048, 224), (2048, 256), (3072, 256)

Information Security

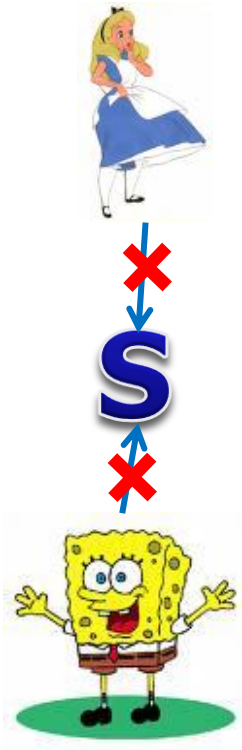


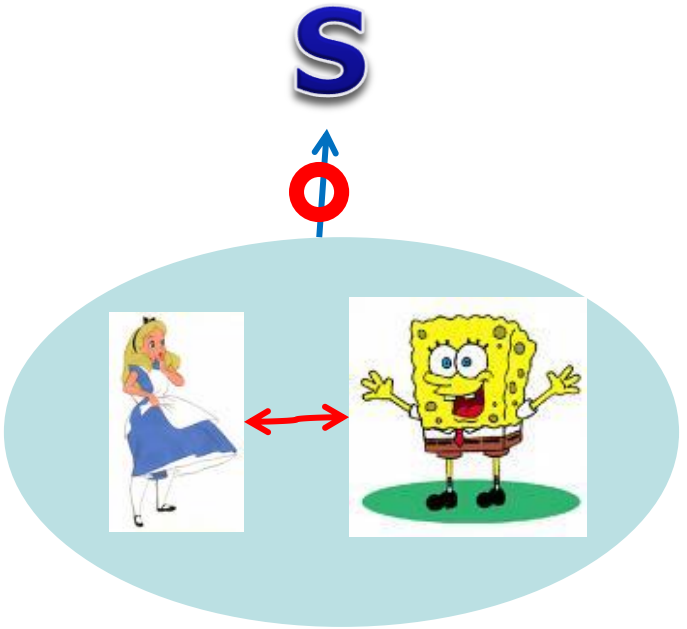
Other topics

AI컴퓨터공학부
김 희 열

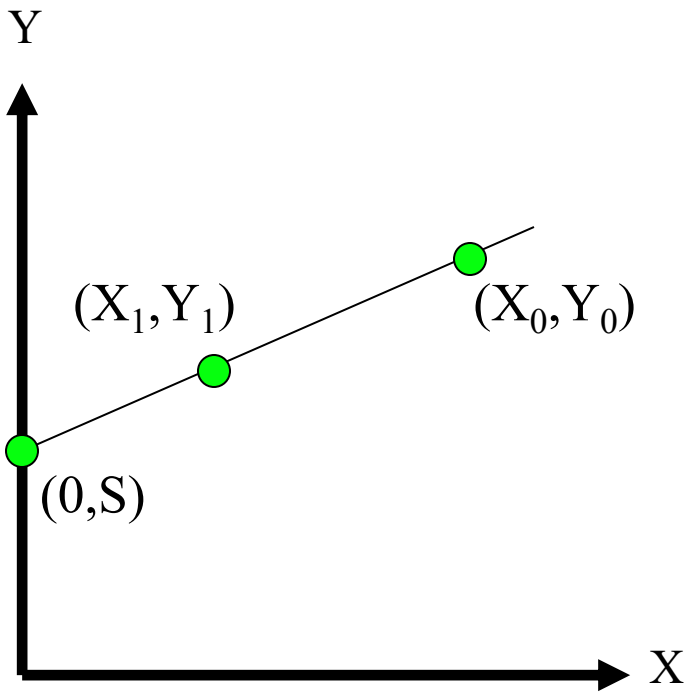
Secret Sharing

- We have a secret S
- We want
  - Neither Alice nor Bob alone cannot get S
  - Alice and Bob together can get S





Shamir's Secret Sharing



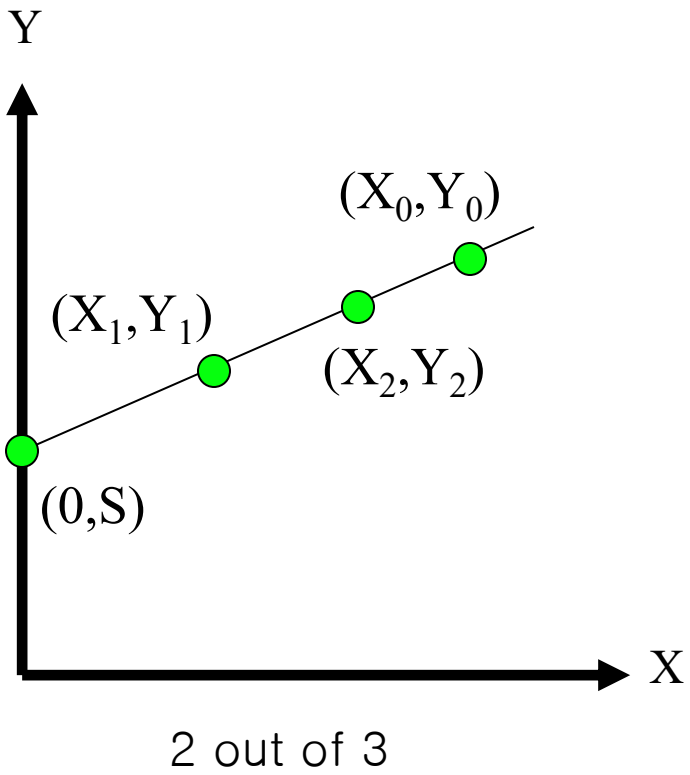
2 out of 2

- Two points determine a line
- Give (X_0,Y_0) to Alice
- Give (X_1,Y_1) to Bob

- Alice and Bob cooperate to find secret S

Easy to make "m out of n" scheme
for any $m \leq n$

Shamir's Secret Sharing



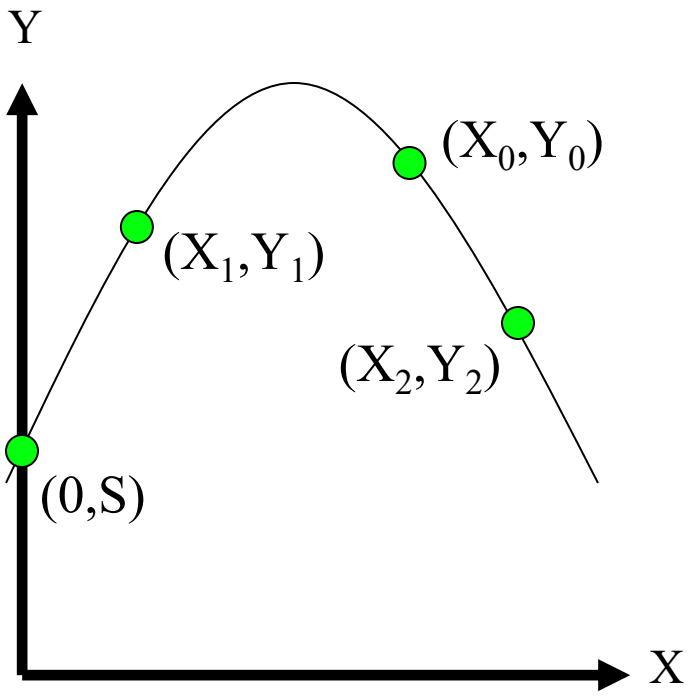
2 out of 3

- Give (X_0,Y_0) to Alice
- Give (X_1,Y_1) to Bob
- Give (X_2,Y_2) to Charlie

- Alice and Bob can cooperate to find S
- Alice and Charlie can cooperate to find S
- Bob and Charlie can cooperate to find S
- But no one can find secret S

- "2 out of 3" scheme

Shamir's Secret Sharing



3 out of 3

- Give (X_0,Y_0) to Alice
- Give (X_1,Y_1) to Bob
- Give (X_2,Y_2) to Charlie
- 3 points determine a parabola
- Alice, Bob and Charlie must cooperate to find secret S

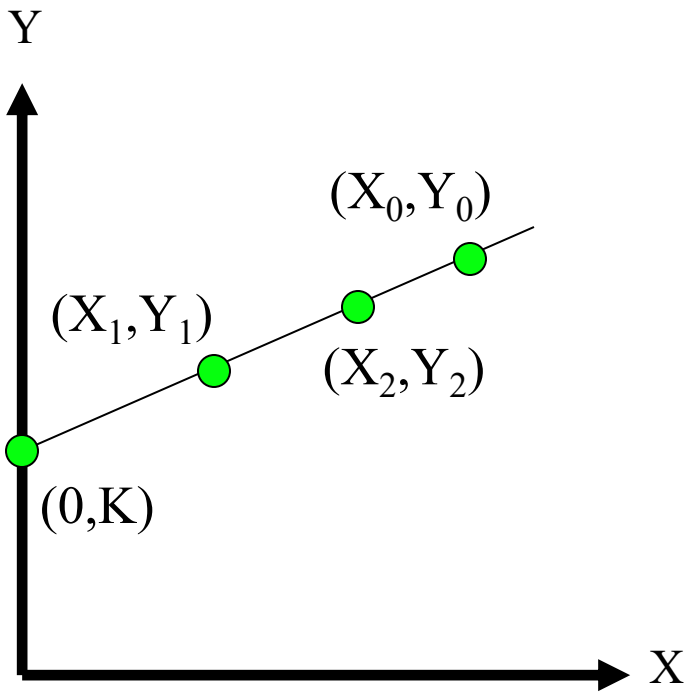
- A "3 out of 3" scheme
- Can you make a "3 out of 4"?

Secret Sharing Example

- **Key escrow**
  - Required that your key should be stored somewhere
- Key can be used with court order
- But you don't trust FBI to store keys

- We can use secret sharing
  - Say, three different government agencies
  - Two must cooperate to recover the key

Secret Sharing Example



- Your symmetric key is K
- Point (X_0, Y_0) to FBI
- Point (X_1, Y_1) to DoJ
- Point (X_2, Y_2) to DoC

- To recover your key K,
  two of the three agencies must cooperate

- No one agency can get K

Random Numbers

- Random numbers used to generate **keys**
  - Symmetric keys
  - RSA: Prime numbers
  - Diffie Hellman: secret values

- Random numbers used for nonces
  - Sometimes a sequence is OK
  - But sometimes nonces must be random

- Random numbers also used in
  - Simulations, statistics, etc.,
  - These numbers only need to be “statistically” random

Random Numbers

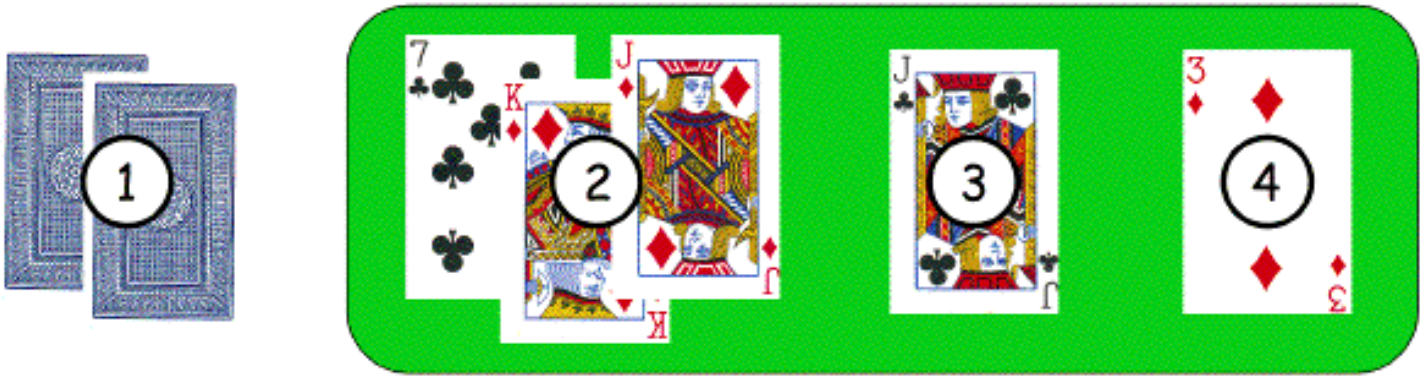
- Cryptographic random numbers must be statistically random and **unpredictable**

- Suppose key server generates symmetric keys
  - Alice: K_A
  - Bob: K_B
  - Charlie: K_C
  - Dave: K_D
- Alice, Bob and Charlie don't like Dave

- Alice, Bob and Charlie working together must **not** be able to determine K_D

Bad Random Number Example

- Online version of Texas Hold'em Poker



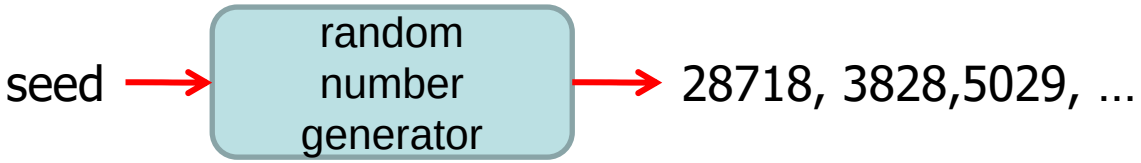
Player's hand

Community cards in center of the table

- Random numbers used to shuffle the deck
- Program did not produce a random shuffle
- Could determine the shuffle in real time!

Card Shuffle

- There are $52! > 2^{225}$ possible shuffles
- The poker program used "random" 32-bit integer to determine the shuffle
  - Only 2^{32} distinct shuffles could occur



- Used Pascal pseudo-random number generator (PRNG): Randomize()
- Seed value for PRNG was function of number of milliseconds since midnight
- Less than 2^{27} milliseconds in a day
  - $24*60*60*1000 < 2^{27}$
  - Therefore, less than 2^{27} possible shuffles

Card Shuffle

- Seed based on milliseconds since midnight
- PRNG re-seeded with each shuffle
- Synchronizing clock with server
  - number of shuffles that need to be tested $< 2^{18}$
- Could try all 2^{18} in real time
  - Test each possible shuffle against “up” cards

- Attacker knows **every card** after the first of five rounds of betting!

Poker Example

- Poker program is an extreme example
  - But common PRNGs are predictable
- Crypto random sequence is not predictable
  - For example, key stream from RC4 cipher
  - But “seed” (or key) selection is still an issue!
- How to generate initial **random** values?

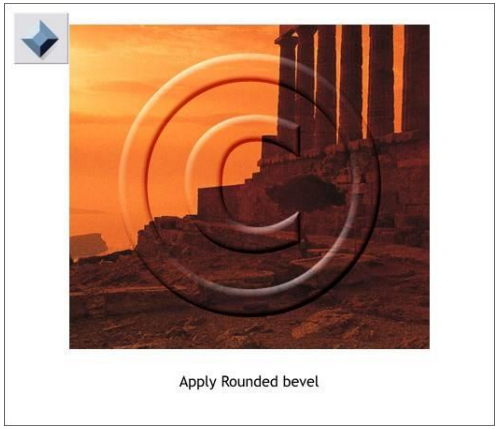
Randomness

- True randomness is hard to define
- **Entropy** is a measure of randomness
- Good sources of “true” randomness
  - Radioactive decay
    - Quantum computers are not too popular
  - Hardware devices
    - Many good ones on the market
  - Human behavior
    - Mouse movements, keyboard dynamics, etc
- Sources of randomness via software
  - Software is deterministic
  - So must rely on external “random” events

pseudo random process for secret → pseudo-security

Information Hiding

- Digital Watermarks
  - Hide identifying information to identify responsible for illegal redistribution
  - Defense against music or software piracy



- Steganography
  - Hide the fact that information is being transmitted
  - Secret communication channel (**covert channel**)

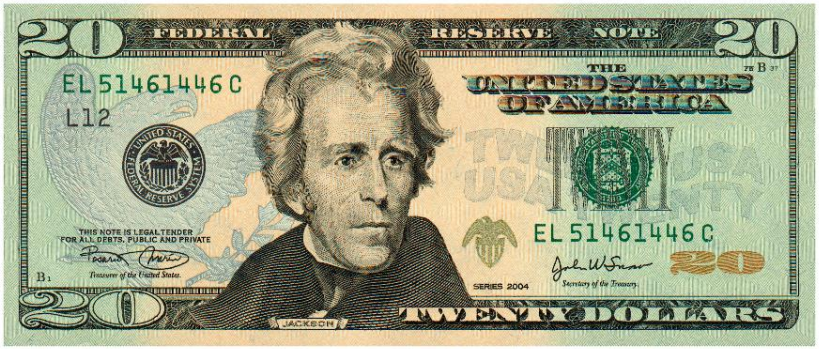
Watermark

- Add a “mark” to data
- Several types of watermarks
  - Invisible — Not obvious the mark exists
  - Visible — Such as **TOP SECRET**
  - Robust — Readable even if attacked
  - Fragile — Mark destroyed if attacked

- Add **robust invisible** mark to digital music
  - If pirated music appears on Internet, can trace it back to original source
- Add **fragile invisible** mark to audio file
  - If watermark is unreadable, recipient knows that audio has been tampered (integrity)
- Combinations of several types are sometimes used
  - E.g., visible plus robust invisible watermarks

Watermark Example

- US currency includes watermark
  - Hold bill to light to see embedded info



- Add **invisible** watermark to photo print
- 1 square inch can contain enough info to reconstruct entire photo
- If photo is damaged, watermark can be read from an undamaged section and entire photo can be reconstructed!

Steganography

- According to Herodotus (Greece 440 BC)
  - Shaved slave's head
  - Wrote message on head
  - Let hair grow back
  - Send slave to deliver message
  - Shave slave's head to expose message (warning of Persian invasion)

- Historically, steganography has been used more than cryptography!

Images and Steganography

- Images use 24 bits for color: **RGB**
  - 8 bits for red, 8 for green, 8 for blue

- For example
  - **0x7E 0x52 0x90** is this color
  - **0xFE 0x52 0x90** is this color
- While
  - **0xAB 0x33 0xF0** is this color
  - **0xAB 0x33 0xF1** is this color

- Low-order bits are unimportant!

Images and Stego

- Given an uncompressed image file
  - For example, BMP format
- Then we can insert any information into low-order RGB bits

- Result will be “invisible” to human eye
- But a computer program can “see” the bits

plain Alice image

With “Alice in Worderlang” PDF data

Non-Stego Example

- Walrus.html in web browser

"The time has come," the Walrus said,
"To talk of many things:
Of shoes and ships and sealing wax
Of cabbages and kings
And why the sea is boiling hot
And whether pigs have wings."

- Source code

```
<font color="#000000">"The time has come," the Walrus said,</font><br>
<font color="#000000">"To talk of many things:</font><br>
<font color="#000000">Of shoes and ships and sealing wax</font><br>
<font color="#000000">Of cabbages and kings</font><br>
<font color="#000000">And why the sea is boiling hot</font><br>
<font color="#000000">And whether pigs have wings."</font><br>
```

Stego Example

- stegoWalrus.html in web browser

"The time has come," the Walrus said,
"To talk of many things:
Of shoes and ships and sealing wax
Of cabbages and kings
And why the sea is boiling hot
And whether pigs have wings."

- Source code

```
<font color="#010100">"The time has come," the Walrus said,</font><br>
<font color="#000100">"To talk of many things:</font><br>
<font color="#010100">Of shoes and ships and sealing wax</font><br>
<font color="#000101">Of cabbages and kings</font><br>
<font color="#000000">And why the sea is boiling hot</font><br>
<font color="#010001">And whether pigs have wings."</font><br>
```

Hidden message : 110 010 110 011 000 101

Steganography

- Some formats (jpg, gif, wav, etc.) are more difficult (than html) for humans to read

- Easy to hide information in **unimportant bits**
- But, easy to **destroy** or remove info stored in unimportant bits!

- To be robust, information must be stored in **important bits**
- But, stored information must not damage data!

- Robust steganography is trickier than it seems

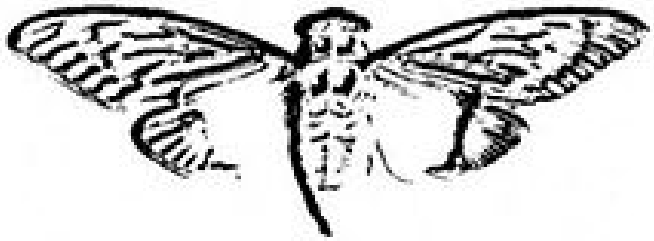
Steganography Examples

Example

Since everyone can read, encoding text in neutral sentences is doubtfully effective

***S**ince **E**veryone **C**an **R**ead, **E**ncoding **T**ext*
***I**n **N**eutral **S**entences **I**s **D**oubtfully **E**ffective*

'Secret inside'



Cicada 3301

https://www.youtube.com/watch?time_continue=1063&v=I2O7bISSzpI&feature=emb_logo

Information Hiding : The Bottom Line

- Surprisingly difficult to hide digital information
- If information hiding is suspected
  - Attacker can probably make information/watermark unreadable
  - Attacker may be able to read the information (Collusion attack)
    - Given the original document (image, audio, etc.)
    - Trudy compares the original object with a watermarked object
- Information hiding is an active research area